

Fluxos orquestrados

`/flow` e `/bug-flow` — como usar, o que cada gate verifica e como a cadeia se comporta quando os agents discordam.

Documento	Guia de uso com exemplos comentados
Público	Engenharia — devs, tech leads, PO e QA
Fonte	<code>commands/flow.md</code> · <code>commands/bug-flow.md</code> · <code>commands/code-review.md</code>
Versão em texto	<code>docs/FLOWS.md</code>
Data	17 de agosto de 2026

Sumário

- 1. Qual fluxo usar
e o que muda entre `/flow`, `/bug-flow`, `/feature-flow` e `/incident-response`
- 2. Pré-requisitos
memória do projeto, confirmação de intenção, ID da demanda, sessões peer
- 3. `/flow` — cadeia e gates
os sete gates, um a um, com o critério exato de aprovação
- 4. Exemplo comentado de `/flow`
demanda real do início ao SHIP, com a consulta peer no meio
- 5. `/bug-flow` — cadeia e gates
nove fases, fast path S1 e o critério de RCA específica
- 6. Exemplo comentado de `/bug-flow`
de um bug intermitente ao teste de regressão que o pinou
- 7. Modelo de looping e discordância
as quatro formas de discordância, a regra do loop-back e o mapa de retorno
- 8. Precedência entre agents
quem ganha quando dois specialists apontam para lados opostos
- 9. Anti-padrões e cheat sheet

1. Qual fluxo usar

Situação	Command	Por quê
Ideia nova, mudança de produto, feature	<code>/flow</code>	Ancora a entrega numa métrica de sucesso antes de qualquer linha de código
Comportamento errado num sistema que já existe	<code>/bug-flow</code>	Prioriza RCA específica e teste de regressão
Feature, mas a métrica já está definida fora	<code>/feature-flow</code>	Mesma cadeia sem a fase GOAL — <i>superseded</i> , prefira <code>/flow</code>
Produção quebrada agora	<code>/incident-response</code>	Mitiga primeiro, investiga depois; cai em <code>/bug-flow</code> para o fix permanente

A única diferença entre `/flow` e `/feature-flow` é a fase 0. Se você não consegue dizer qual número muda com a entrega, o `/flow` te obriga a descobrir isso antes — e esse é o ponto dele.

2. Pré-requisitos

Os dois fluxos abrem com o mesmo pre-flight:

- 1. **Memória do projeto carregada.** Precisa existir `.claude/memory/{business,architecture,guidelines}.md`. Se faltar, o fluxo **para** e pede `/bootstrap-project`.
- 2. **Intenção confirmada.** O orquestrador reformula o pedido em uma frase e confirma com você antes de gastar tokens na cadeia.

3. **ID da demanda alocado.** Próximo `NNN` varrendo `docs/todo/` e `docs/done/` ; cria `docs/todo/<NNN>-<nome>/task.md` (flow) ou `bug.md` (bug-flow).
4. **Sessões peer (só /flow , opcional).** `ListAgents` → uma sessão por repo afetado → handshake. Sem peer aberto, a cadeia roda idêntica.

O orquestrador nunca escreve código

Ele delega para o agent da stack, avalia o gate e sintetiza. Isso vale para os dois fluxos, em todas as fases.

3. /flow — a cadeia GOAL → SHIP

/flow reduzir o tempo de primeira resposta do suporte incluindo histórico de pedidos na tela do atendente

/flow — GOAL → SHIP

Sete fases, sete gates. Nada avança com gate vermelho.



a métrica GOAL da fase 0 é referenciada por todas as outras — fase que não se amarra nela é sinal de re-scope

Cada seta entre fases é um gate. Gate que não está verde não passa.

A métrica GOAL estabelecida na fase 0 é o fio que atravessa todas as outras: toda fase seguinte carrega essa métrica no seu briefing, e a fase que não consegue se amarrar nela é sinal de re-scope, não de seguir em frente.

3.1 Cada gate, em detalhe

0 GOAL — Discovery

Roda	<code>/discovery</code>
Produz	<code>docs/discovery/<slug>.md</code> — problem statement, usuários afetados, outcome JTBD e a métrica de sucesso
Verde quando	recomendação é Go e a métrica é concreta: nome + baseline com fonte + alvo com data
Amarelo	Learn-more → pausa o fluxo até os experimentos sinalizados rodarem
Vermelho	No-go → encerra o fluxo

“Reduzir tempo de resposta” **não** passa. “TTFR mediano do suporte: 8m40s (baseline: dashboard support-ops, jul/2026) → 4m até 30/09” passa.

1 DEFINE — Spec / PO

Roda	<code>senior-product-owner</code>
Produz	em <code>task.md</code> : descrição, segmento, stories INVEST, AC em Gherkin, out of scope e a ligação AC ↔ métrica GOAL
Verde quando	AC testáveis · escopo delimitado · sem conflito com <code>business.md</code> · ao menos uma AC move a métrica GOAL
Memória	regra de domínio nova → <code>project-memory-keeper</code> atualiza <code>business.md</code>

A cláusula que mais reprova aqui é a última: um conjunto de AC bem escrito que não move a métrica declarada na fase 0 significa que o escopo derivou.

2 PLAN — Arquitetura

Roda	<code>system-architect</code> (+ <code>integration-architect</code> se integração-pesado, + <code>postgres-dba</code> se DB-pesado)
Produz	guia de implementação, trade-offs, abordagem recomendada, decomposição em tarefas atômicas mapeadas a AC, ADR se houver decisão estrutural
Verde quando	tarefas atômicas e ordenadas por dependência · impacto de NFR não ameaça a métrica GOAL · superfície sensível marcada
Sensível	auth, authz, sessão, secrets, PII, pagamentos, upload/download, desserialização, SQL cru, shell exec, isolamento multi-tenant, nova integração externa

Consulta peer (condicional). Se o design muda um contrato que outro repo consome e há sessão aberta lá, o `/flow` pergunta o que aquele repo *de fato* consome hoje — ancorado em `origin/main`, exigindo que a resposta cite ref e sha. Contradição aqui vale um re-scope **antes** do BUILD.

2.5 SEC — Threat modeling (condicional)

Roda	<code>security-specialist</code> — só se a fase 2 marcou superfície sensível. Obrigatória quando cruza trust boundary
Produz	tabela STRIDE + lista de “controles obrigatórios” na seção Security do <code>task.md</code>
Verde quando	os controles são concretos e atribuídos ao BUILD

“Validar entrada” não é controle concreto. “Rejeitar `order_id` que não pertença ao `tenant_id` da sessão, com teste em `handler_test.go`” é.

3 BUILD — Dev por stack

Roda	Go → go-senior-engineer · .NET → dotnet-backend-architect · Node/TS → nodejs-backend-architect · Python → python-engineer · React → senior-react-developer · Postgres-pesado → postgres-dba junto · IaC → aws-devops-engineer
Branch	feat/<nome> ou feat/<NNN>-<nome>
Produz	código + testes unitários, uma fatia atômica por vez , seguindo <code>guidelines.md</code> ; instrumenta a métrica GOAL se ela ainda não for observável
Verde quando	toda AC é verificável no diff · lint e unit verdes localmente · métrica instrumentada ou já observável

4 VERIFY — QA/SDET + gate de segurança

Roda	go-sdet-backend (Go) · cypress-qa-analyst (front/E2E) · os dois quando a mudança atravessa · depois, sempre, security-specialist
Produz	testes de integração/E2E/fuzz, matriz AC → teste, confirmação de que a métrica emite
Verde quando	toda AC tem ao menos um teste que falha sem a mudança · a métrica emite e é consultável · veredicto APPROVE ou APPROVE WITH MITIGATIONS
Vermelho	BLOCK → volta pro BUILD

O critério “falha sem a mudança” é literal: um teste que passa tanto com quanto sem o patch não conta como cobertura de AC.

5 REVIEW

Roda	<code>/code-review</code> — dispara em paralelo os especialistas aplicáveis ao diff (máx. 6), agrega e deduplica
Verde quando	nenhum BLOCKER
Cuidado	NEEDS DISCUSSION significa que dois reviewers se contradisseram — decisão humana antes de qualquer retorno

6 SHIP

Roda	<code>/smart-commit</code> + fechamento
Ações	<code>project-memory-keeper</code> sincroniza a memória com o que de fato saiu (não com o que foi planejado) · registra o baseline da métrica no momento do ship · notifica peers de repos afetados (só notificação) · move <code>docs/todo/</code> → <code>docs/done/</code>
Verde quando	commits criados · memória sincronizada · baseline registrado · demanda movida

4. Exemplo comentado — /flow

```
/flow reduzir o tempo de primeira resposta do suporte incluindo
histórico de pedidos na tela do atendente
```

Pre-flight

```
Memória carregada: business.md, architecture.md, guidelines.md ✔
Entendi: expor os últimos pedidos do cliente na tela do atendente pra cortar
o tempo até a primeira resposta. Confirma? → sim
Demanda alocada: docs/todo/047-historico-pedidos-atendente/task.md
Peers: orders-api-3f (handshake ok – git@github.com:ems/orders-api.git,
main, 9c1a4e2, clean)
```

Fase 0 — GOAL

```
[047-historico-pedidos-atendente] GOAL – /discovery – PASS
Produced: docs/discovery/historico-pedidos-atendente.md
GOAL metric: TTFR mediano do suporte
8m40s (support-ops dashboard, jul/2026) → 4m até 30/09
Recommendation: Go
Next: DEFINE – senior-product-owner
```

Fase 1 — DEFINE

O PO devolve 3 stories e 6 AC. Uma delas:

```
Scenario: atendente abre um ticket de cliente com pedidos recentes
  Given um cliente com ao menos 1 pedido nos últimos 90 dias
  When o atendente abre o ticket
  Then os 5 pedidos mais recentes aparecem no painel lateral em até 500ms
# Moves GOAL metric? sim – remove a troca de aba pro admin, que é o maior
# bloco de tempo medido antes da primeira resposta
```

Fase 2 — PLAN, e a consulta peer

O arquiteto propõe um endpoint novo no `orders-api` consumido pelo front do suporte. Contrato cross-repo → dispara a consulta peer:

```
SendMessage → orders-api-3f:
  "Vou adicionar GET /v1/customers/{id}/orders?limit=5 no orders-api.
  Responda a partir de origin/main – não da sua working tree.
  Existe endpoint equivalente hoje? Qual o shape de OrderSummary e quais
  campos são obrigatórios? Cite ref, sha curto, arquivo e linha.
  Não altere nada."

← resposta (origin/main@9c1a4e2, internal/api/orders.go:88):
  "Já existe GET /v1/orders?customer_id=&limit= com o mesmo shape.
  OrderSummary tem id, status, total_cents, created_at – todos obrigatórios."
```

Por que isso importa

O plano muda: reusar o endpoint existente em vez de criar um novo. Uma tarefa a menos e zero mudança de contrato. Um subagent aqui só poderia *inferir* o que o outro repo consome; a sessão que está dentro dele foi *ler*. É o único caso em que um peer bate um subagent.

```
[047-...] PLAN - system-architect - PASS
Produced: task.md §5, ADR-0012 (cache do painel lateral)
GOAL metric: TTFR - traceable? yes (NFR de 500ms é a AC #3)
Peer consult: orders-api-3f → endpoint já existe; plano de 5 pra 4 tarefas
Sensitive surface: PII (dados de pedido de cliente) → SEC obrigatória
Next: SEC - security-specialist
```

Fases 2.5 a 5

- **2.5 SEC** — o STRIDE aponta *Information Disclosure*: o front do suporte poderia pedir pedidos de qualquer `customer_id`. Controle obrigatório atribuído ao BUILD: checagem de escopo do atendente + teste.
- **3 BUILD** — `nodejs-backend-architect` no front, `go-senior-engineer` no consumo do `orders-api`. Instrumenta o timer `support.ttfr`.
- **4 VERIFY** — 6 AC, 6 testes, todos falhando sem o patch. O gate de segurança valida que o controle da 2.5 aterrissou no diff. O peer confirma que o consumo não quebrou.
- **5 REVIEW** — `/code-review` devolve 1 WARNING (cache sem TTL explícito) e zero BLOCKER → passa.

Fase 6 — SHIP

```
[047-historico-pedidos-atendente] SHIPPED
branch: feat/047-historico-pedidos-atendente
Phases:  GOAL ✓ DEFINE ✓ PLAN ✓ (SEC ✓) BUILD ✓
          VERIFY ✓ REVIEW ✓ SHIP ✓
GOAL:    TTFR mediano - baseline 8m40s → target 4m até 30/09
          (baseline@ship: 8m31s)
Files:   7 created, 11 modified
ADRs:    ADR-0012
AC:      6/6 covered
Security: APPROVE
Peers:   1 consulted, 1 repo notified
Follow-ups: medir TTFR em 14/09 e comparar com o baseline@ship
legend:  ✓ pass · - skipped (not triggered) · ✗ blocked
```

O `baseline@ship` é o que torna a métrica auditável depois: sem ele, daqui a três semanas ninguém sabe se os 8m40s de julho ainda eram verdade no dia do merge.

5. /bug-flow — do report ao fix merged

/bug-flow autovacuum não roda em public.events e bloat passou de 40%
– ver thread no Slack #db

/bug-flow — report → fix merged

Toda fix sai com um teste que falha antes e passa depois. Sem exceção.



Nove fases. A 6 é condicional; a 3 é implícita no fast path S1.

5.1 Os gates que mais reprovam

Gate 2 — RCA específica

Este é o gate mais reprovado do fluxo, e o critério é literal:

❌ Não passa	✅ Passa
“problema de concorrência”	<code>internal/cache/cache.go:142</code> — compare-and-swap não atômico com $N \geq 2$ escritores
“o autovacuum está mal configurado”	<code>public.events</code> tem <code>autovacuum_vacuum_scale_factor=0.2</code> herdado do default; com 40M linhas o threshold só dispara a cada 8M tuplas mortas

Se o bug é instância de um anti-padrão que já está em `guidelines.md`, isso é uma **regressão** e precisa ser sinalizada como tal. Se é classe nova, o `project-memory-keeper` registra depois do fix.

Gate 3 — Severidade e superfície sensível

Sev	Significa	Caminho
S1	prod fora, perda de dados, brecha de segurança, dinheiro em risco	hotfix (fast path)
S2	fluxo core quebrado para muitos usuários, sem workaround confiável	normal, prioridade alta
S3	fluxo não-core degradado, com workaround	normal
S4	cosmético / edge case / sem impacto visível	normal

Bug em ou perto de auth, authz, secrets, PII, pagamentos, manipulação de arquivo, desserialização, SQL cru ou isolamento multi-tenant torna a fase 6 **obrigatória**.

Gate 4 — Fix com teste que pina o bug

Verde quando a causa raiz foi atacada (não o sintoma), existe **teste de regressão que falha antes do fix e passa depois**, e **nenhuma refatoração de carona** entrou no diff. O teste é verificado revertendo o fix e confirmando que ele fica vermelho.

Fast path S1

`1 → 2 → 4 → 5 → (6 se aplicável) → 7 → 9`. A fase 3 é implícita — declarar S1 já classificou a severidade. A fase 7 continua obrigatória, mas abreviada a BLOCKERs. A fase 8 não bloqueia o fix e também não é pulada: roda em até 24h pós-merge.

6. Exemplo comentado — /bug-flow

```
/bug-flow o contador de itens no carrinho às vezes fica menor que o número
real depois de dois adds rápidos – intermitente em staging,
sem stack trace
```

Fase 1 — Triage

```
[052-carrinho-contador-perdido] Phase 1 – orchestrator – PASS
Repro: 2 POST /cart/items concorrentes no mesmo cart_id → contador
       incrementa 1x (10 execuções, falha em 6). Documentado em bug.md §2.
Suspeita: internal/cart/counter.go (read-modify-write sem lock)
Next: Phase 2 – go-senior-engineer
```

Fase 2 — RCA

```
Defect: internal/cart/counter.go:74 – `c.count = c.count + n` sob RWMutex
       tomado em RLock (leitura) e não em Lock (escrita). Dois writers
       concorrentes leem o mesmo valor e escrevem o mesmo resultado;
       um incremento é perdido.
Por que não foi pego: TestCounterAdd é sequencial; não há teste com -race.
Hipóteses descartadas: retry do cliente (logs mostram 2 requests distintos,
       IDs diferentes); cache stale (o valor errado persiste no banco).
```

Específico, com evidência e `arquivo:linha` → **gate verde**.

Fase 3 — Severidade

S2: fluxo core, sem workaround, o cliente perde item silenciosamente. Superfície sensível: **nenhuma** → a fase 6 não roda.

Fase 4 — Fix (teste primeiro)

```
func TestCounterAdd_Concurrent(t *testing.T) { // falha com -race antes do fix
    c := NewCounter()
    var wg sync.WaitGroup
    for i := 0; i < 100; i++ {
        wg.Add(1)
        go func() { defer wg.Done(); c.Add(1) }()
    }
    wg.Wait()
    if got := c.Value(); got != 100 { t.Fatalf("got %d, want 100", got) }
}
```

O fix é trocar `RLock` por `Lock` no caminho de escrita: 1 arquivo, 2 linhas. Revertido o fix, o teste fica vermelho — **gate verde**.

Fase 5 — QA amplia o escopo do fix

O `go-sdet-backend` nota que a RCA aponta uma *classe*: todos os writers do pacote `cart`. Encontra o mesmo padrão em `internal/cart/reservations.go:51` e adiciona teste + fix. A suíte passa a rodar com `-race` no CI.

Fechamento

```
[052-carrinho-contador-perdido] DONE
branch: fix/052-carrinho-contador-perdido - severity: S2
RCA:      RLock usado em caminho de escrita em counter.go:74;
          incremento perdido sob concorrência
Files:    4 changed
Tests:    regressão em counter_test.go + reservations_test.go;
          CI agora roda -race
Security: n/a
Memory:   guidelines.md (anti-padrão RWMutex)
Follow-ups: varrer os outros pacotes pelo mesmo padrão (task 053)
```

O que este exemplo mostra

- A RCA nomeia `arquivo:linha` e o passo em falso — é isso que libera o gate 2, não a hipótese plausível.
- O teste veio antes do fix e foi validado ao contrário: revertendo o patch, ele fica vermelho.
- O QA não só conferiu o teste: leu a RCA como *classe* de defeito e achou o mesmo padrão noutro arquivo.
- A memória guardou o anti-padrão, não o incidente — é o que impede a terceira ocorrência.

7. Modelo de looping e discordância

Um gate reprovado **não** é um erro do fluxo — é o fluxo funcionando. O que importa é para onde a cadeia volta, quem decide e como ela não gira em círculo.

Quando um gate reprova

Nenhum loop é automático: o orquestrador para, propõe o menor caminho pro verde e pede confirmação.



regra do loop-back: volte para quem PRODUZIU o defeito, não para a fase imediatamente anterior

Os cinco desfechos de um gate. Nenhum retorno acontece sem confirmação humana.

As quatro formas de discordância

1. **O gate reprova o artefato da própria fase.** AC não testável, RCA vaga, teste que não falha sem o fix. Ninguém discordou de ninguém — a fase simplesmente não entregou o que o gate pede. A fase *reexecuta*, não volta.
2. **O gate reprova por causa de uma decisão tomada lá atrás.** Security dá BLOCK no VERIFY porque o *design* da fase 2 é que é inseguro. Aqui a cadeia volta de verdade — e volta para a fase 2, não para a 3.
3. **Dois agents na mesma fase se contradizem.** É o caso do `/code-review`, que roda reviewers em paralelo. Mesmo `file:line` e mesmo problema → deduplica e mantém a severidade mais estrita. Recomendações genuinamente opostas → o command **não escolhe lado**: o achado vai para *Conflitos entre reviewers* e o veredicto vira **NEEDS DISCUSSION**.
4. **Um peer contradiz o design.** A resposta do peer é **evidência, não veredicto** — vem de um contexto que você não enxerga. Se ela muda uma decisão, confirme contra algo concreto antes de agir. E peer nunca segura gate: peer calado vira “não respondido” e a cadeia segue.

A regra do loop-back

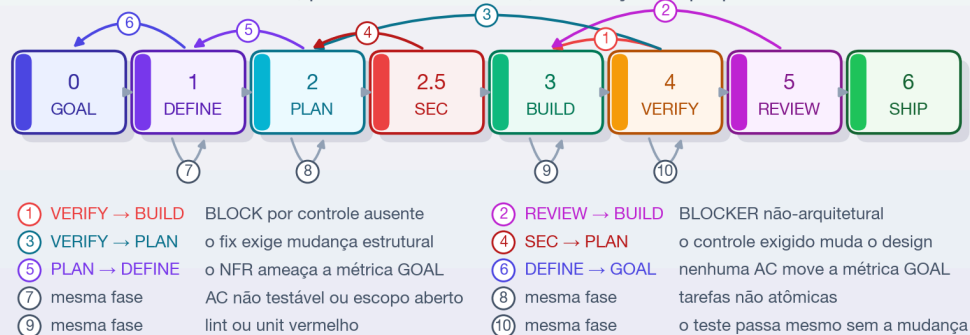
Volte para quem produziu o defeito, não para a fase imediatamente anterior

Um BLOCK de segurança no VERIFY causado por controle ausente volta pro BUILD. O mesmo BLOCK causado por um modelo de dados que expõe PII por construção volta pro PLAN — e obriga a 2.5 a revalidar. Voltar pro BUILD nesse segundo caso só produz um patch em cima de um design que continua errado.

Reexecutar uma fase reexecuta os gates a jusante *que foram afetados*. Consertar um teste no VERIFY não obriga a refazer o PLAN; mudar o contrato no PLAN obriga a refazer tudo depois dele. E o loop nunca é automático: em **BLOCK** ou **No-go** o orquestrador para, mostra o achado, propõe o menor caminho pro verde e pede sua confirmação antes de tentar de novo.

Mapa de retorno

De onde falha, pra onde volta. Abaixo, reexecução da própria fase.



O gate que reprovou não decide pra onde volta — quem decide é onde o defeito NASCEU.

BLOCK no VERIFY por controle ausente volta pro BUILD. O mesmo BLOCK por um modelo de dados que expõe PII por construção volta pro PLAN.

Os seis retornos entre fases e as quatro reexecuções no lugar.

Matriz de retorno — /bug-flow

Gate	Achado	Volta pra
1 Triage	não reproduz e não há dados	1 — marca “não-reproduzível” e pede dados ao reporter
2 RCA	causa vaga, ou causa que é sintoma de outra mais funda	2
4 Fix	trata sintoma; teste não vira vermelho ao reverter; refactor de carona	4
5 QA	colateral quebrado	4
5 QA	RCA aponta classe subtestada	4 — amplia o fix
6 Security	BLOCK	4 (explícito no command)
7 Review	BLOCKER	4
7 Review	NEEDS DISCUSSION	humano decide antes de qualquer volta

8. Precedência quando dois agents discordam

Precedência quando dois agents discordam

Da regra mais forte para a mais fraca. O orquestrador nunca escolhe lado sozinho.

1 Security BLOCK para tudo

Não é negociável por outro agent. A cadeia só reentra depois do finding tratado.

2 BLOCKER de review é corrigido antes do merge

Sem exceção. WARNING e INFO não seguram o gate; BLOCKER sim.

3 Convenção do projeto > best practice genérica

Padrão que o codebase usa consistentemente não é achado, venha de qual specialist vier. guidelines.md é o árbitro.

4 Evidência > inferência

Peer citando arquivo:linha num ref ancorado ganha do arquiteto que infere. Sem ref e sha, é resposta sobre working tree.

5 Contradição que evidência não resolve → humano

O orquestrador registra os dois lados em NEEDS DISCUSSION e para. Você decide, e só então define o retorno.

Peer session nunca é dono de gate

Peer informa o gate, nunca o segura. Sem resposta → “não respondido” e a cadeia segue. E nunca peça a um peer o que esta sessão teve negado.

Da regra mais forte para a mais fraca. O orquestrador nunca escolhe lado sozinho.

Convenção anti-loop-infinito

Convenção de uso, não regra escrita nos commands

Os commands definem o retorno e a confirmação humana, mas não um contador de tentativas. O que segue é a convenção recomendada — se quiser que valha como regra, dá para codificar nos próprios commands.

- **Duas voltas no mesmo gate sem ficar verde → pare.** A terceira tentativa quase nunca é o problema; ele costuma estar uma fase acima. Escale para o humano ou re-escope.
- **Registre cada volta** no `task.md` / `bug.md`, com fase, achado e o que mudou na tentativa. Sem isso ninguém enxerga que já foram três voltas.
- **Volta que sobe de fase reinicia a contagem.** Voltar do VERIFY pro PLAN é um caminho novo, não a terceira tentativa do mesmo.
- **NEEDS DISCUSSION não consome tentativa.** Ele suspende a cadeia esperando decisão humana; nada foi tentado ainda.

9. Anti-padrões

✗	Por quê
Pular fase “pra ganhar tempo”	Os gates são o valor. Cadeia sem gate é só delegação em série
Rodar <code>/flow</code> sem métrica com baseline	A fase 0 para de propósito. Sem métrica você não está pronto pra especificar
O orquestrador escrever código	Ele delega, avalia gate e sintetiza. Código é do dev da stack
Pedir a um peer o que esta sessão teve negado	Contorna uma decisão que <i>você</i> tomou. Trabalho bloqueado volta pro usuário
Perguntar pra três sessões do mesmo repo	Não são três fontes — é um repo visto de três checkouts. Uma por repo, ancorada em ref
Deixar um gate esperando peer	Peer informa gate, nunca é dono dele. Sem resposta → “não respondido” e segue
Refatoração de carona no <code>/bug-flow</code>	Bug fix é uma mudança só. Limpeza vai pro <code>/flow</code> ou task própria
Aceitar RCA vaga	“Problema de concorrência” não é causa raiz; é a descrição do sintoma
Escolher lado num NEEDS DISCUSSION	O conflito existe porque dois specialists competentes discordaram. É decisão humana

10. Cheat sheet

```
/flow <ideia>
  GOAL → DEFINE → PLAN → (SEC) → BUILD → VERIFY → REVIEW → SHIP

/bug-flow <sintoma + evidência>
  Triage → RCA → Sev → Fix → QA → (Sec) → Review → Memory → Commit

Artefatos
  docs/discovery/<slug>.md          brief da fase GOAL
  docs/todo/<NNN>-<nome>/task.md    demanda de feature, seções 0–10
  docs/todo/<NNN>-<nome>/bug.md     demanda de bug, seções 1–6
  docs/done/<NNN>-<nome>/          pra onde vai no SHIP / fase 9
  docs/adr/ADR-NNNN-*.md           decisões estruturais da fase PLAN
  .claude/memory/{business,architecture,guidelines}.md

Vocabulário de gate
  Go / No-go / Learn-more          fase GOAL
  APPROVE / APPROVE WITH MITIGATIONS security-specialist – verde
  BLOCK                             security-specialist – para a cadeia
  APPROVED / ...WITH COMMENTS      /code-review – verde
  CHANGES REQUESTED               /code-review – tem BLOCKER
  NEEDS DISCUSSION                 /code-review – reviewers se
                                   contradisseram → humano decide

Regra do loop-back
  Volte pra fase que PRODUZIU o defeito, não pra anterior imediata.
  Nenhum loop é automático – o orquestrador propõe e você confirma.
  Duas voltas no mesmo gate → pare e escale (convenção).
```